

Relatório do Projeto 2

Link do GitHub:

<https://github.com/jhapreis/7600017-introducao-fisica-computacional>

Sumário

1	Tarefa 1 - Derivação Numérica	1
1.1	Regras de derivação numérica	1
1.2	Resultados e conclusões	1
2	Tarefa 2 - Integração (ou Quadratura) numérica	3
2.1	Regras de aproximação de integrais	3
2.2	Resultados e conclusões	3
3	Tarefa 3 - Raízes de funções	4
3.1	Métodos numéricos para o cálculo de raízes	4
3.2	Resultados e conclusões	5

1 Tarefa 1 - Derivação Numérica

1.1 Regras de derivação numérica

Partindo da série de Taylor

$$f(x+h) = f(x) + f'(x)h + \frac{1}{2!}f''(x)h^2 + \frac{1}{3!}f'''(x)h + \dots \quad (1)$$

é possível encontrar aproximações para as derivadas. No nosso problema, estamos interessados em aproximações para as derivadas primeira e segunda.

Derivada para frente de dois pontos:

$$f'_{2f}(x) = \frac{f(x+h) - f(x)}{h} + \mathcal{O}(h)$$

Derivada para trás de dois pontos:

$$f'_{2t}(x) = \frac{f(x) - f(x-h)}{h} + \mathcal{O}(h)$$

Derivada simétrica de três pontos:

$$f'_{3s}(x) = \frac{f(x+h) - f(x-h)}{2h} + \mathcal{O}(h^2)$$

Derivada simétrica de cinco pontos:

$$f'_{5s}(x) = \frac{f(x-2h) - 8f(x-h) + 8f(x+h) - f(x+2h)}{12h} + \mathcal{O}(h^4)$$

Derivada segunda simétrica de três pontos:

$$f''_{3s}(x) = \frac{f(x+h) - 2f(x) + f(x-h)}{h^2} + \mathcal{O}(h^2)$$

Derivada segunda simétrica de cinco pontos:

$$f''_{5s}(x) = \frac{-f(x-2h) + 16f(x-h) - 30f(x) + 16f(x+h) - f(x+2h)}{12h^2} + \mathcal{O}(h^4)$$

1.2 Resultados e conclusões

Nosso problema consiste em avaliar a seguinte função e suas derivadas no ponto $x = 0.1$:

$$\begin{aligned} f(x) &= e^{x^2} \cot(2x) = \frac{e^{x^2}}{\tan(2x)} \\ f'(x) &= 2e^{x^2} \left(\frac{x}{\tan(2x)} - \frac{1}{\sin^2(2x)} \right) \\ f''(x) &= e^{x^2} \left(\frac{4x^2}{\tan(2x)} - \frac{8x}{\tan^2(2x)} - 8x + \frac{8}{\tan^3(2x)} + \frac{10}{\tan(2x)} \right) \end{aligned}$$

A partir da análise da Tabela 1, podemos notar que, com valores muito **grandes** de h , o erro de $\mathcal{O}(h^n)$ (o erro da série de Taylor) fica dominante e difere consideravelmente do resultado exato. Porém, ao utilizar

```

1 ! Derivação numérica
2
3 implicit none
4
5 integer i
6 double precision x ! Ponto para cálculo da derivada
7 double precision h ! Tamanho dos passos a cada interação
8 double precision f, f_1, f_2t, f_3s, f_5s, f_1_2f, f_1_3s, f_1_5s
9 double precision f_1_2f, f_1_2t, f_1_3s, f_1_5s, f_1_1_3s, f_1_1_5s
10
11 open(unit=10, file='tarefa1-saida-derivadas-f.dat')
12
13 x = 0.100
14
15 write(10, *) "Valores exatos:"
16 write(10, *) x, f(x), f_1(x), f_1_1(x)
17 write(10, *) "Valores das derivadas numéricas:"
18
19 do i=-1,-12,-1
20   h = 5.000 ** i
21   write(10, *)
22   #
23   # f_1_2f(x, h),
24   # f_1_2t(x, h),
25   # f_1_3s(x, h),
26   # f_1_5s(x, h),
27   # f_1_1_3s(x, h),
28   # f_1_1_5s(x, h)
29 end do
30
31 close(10)
32
33 stop
34 end
35
36 double precision function f(x)
37 implicit none
38 double precision x
39 f = exp(x**2) / tan(2.000*x)
40
41 return
42 end
43
44 double precision function f_1(x)
45 implicit none
46 double precision x
47 f_1 = 2.000 * exp(x**2) * (
48   # x / tan(2.000*x)
49   # - 1.000 / sin(2.000*x)**2
50 )
51 return
52 end
53
54 double precision function f_1_2f(x, h)
55 implicit none
56 double precision x
57 f_1_2f = exp(x**2) * (
58   # 4.000*x**2 / tan(2.000*x)
59   # - 8.000*x / (tan(2.000*x)**2)
60   # - 8.000*x
61   # + 8.000 / tan(2.000*x)**3
62   # + 10.000 / tan(2.000*x)
63 )
64 return
65 end
66
67 ! Derivada f_1(2f) de dois pontos, para frente
68 double precision function f_1_2t(x, h)
69 implicit none
70 double precision f
71 double precision x, h
72 f_1_2t = 1.000 / h * (f(x+h) - f(x))
73 return
74 end
75
76 ! Derivada f_1(2f) de dois pontos, para trás
77 double precision function f_1_2s(x, h)
78 implicit none
79 double precision f
80 double precision x, h
81 f_1_2s = 1.000 / h * (f(x) - f(x-h))
82 return
83 end
84
85 ! Derivada f_1(2f) de três pontos, simétrica
86 double precision function f_1_3s(x, h)
87 implicit none
88 double precision f
89 double precision x, h
90 f_1_3s = 1.000 / (2.000 * h) * (f(x+h) - f(x-h))
91 return
92 end
93
94 ! Derivada f_1(5s) de cinco pontos, simétrica
95 double precision function f_1_5s(x, h)
96 implicit none
97 double precision f
98 double precision x, h
99 f_1_5s = 1.000 / (12.000 * h) * (
100   # f(x - 2.000*h)
101   # - 8.000 * f(x-h)
102   # + 8.000 * f(x+h)
103   # - f(x + 2.000*h)
104 )
105 return
106 end
107
108 ! Derivada f_1(2s) de três pontos, simétrica
109 double precision function f_1_1_3s(x, h)
110 implicit none
111 double precision f
112 double precision x, h
113 f_1_1_3s = 1.000/h**2 * (f(x+h) - 2.000*f(x) + f(x-h))
114 return
115 end
116
117 ! Derivada f_1(2s) de cinco pontos, simétrica
118 double precision function f_1_1_5s(x, h)
119 implicit none
120 double precision f
121 double precision x, h
122 f_1_1_5s = 1.000/(12.000 * h**2) * (
123   # - f(x-2.000*h)
124   # + 16.000*f(x-h)
125   # - 36.000*f(x)
126   # + 16.000*f(x+h)
127   # - f(x+2.000*h)
128 )
129 return
130 end

```

Figura 1: Código utilizado na tarefa 1.

Tabela 1: Derivadas numéricas de $f(x)$ no ponto $x = 0.1$ por meio de diferentes aproximações em função do passo h .

h	$f'_{2f}(x)$	$f'_{2t}(x)$	$f'_{3s}(x)$	$f'_{5s}(x)$	$f''_{3s}(x)$	$f''_{5s}(x)$
5^{-1}	-16.91691895225	49.82733906339	16.45521005557	20.93035784300	-333.72129007817	-422.58598116706
5^{-2}	-35.90741911352	-83.51176602309	-59.70959256831	-33.25352704342	1190.10867273926	661.00897284517
5^{-3}	-46.48255415943	-54.53115014481	-50.50685215212	-50.17628829365	1006.07449817228	999.46407975254
5^{-4}	-49.39764311204	-50.99746610649	-50.19755460926	-50.18473655650	999.88937153697	999.63304482318
5^{-5}	-50.02531878422	-50.34520471938	-50.18526175180	-50.18474965954	999.64354737490	999.63330691480
5^{-6}	-50.15278188421	-50.21675844212	-50.18477016316	-50.18474968048	999.63371738218	999.63330820428
5^{-7}	-50.17835284643	-50.19114815312	-50.18475049978	-50.18474968046	999.63333509250	999.63332650923
5^{-8}	-50.18347018215	-50.18602924414	-50.18474971315	-50.18474968027	999.63358988001	999.63373669905
5^{-9}	-50.18449377239	-50.18500558868	-50.18474968053	-50.18474967866	999.64117929521	999.64823790311
5^{-10}	-50.18469848017	-50.18480086355	-50.18474967186	-50.18474966680	999.83769093898	999.94357005738
5^{-11}	-50.18473939362	-50.18475999347	-50.18474969355	-50.18474969716	1005.85162486448	1008.14567242996
5^{-12}	-50.18474741672	-50.18475197037	-50.18474969355	-50.18474976583	1111.73074327127	1155.84704260743
Exato	-50.18474968051	-50.18474968051	-50.18474968051	-50.18474968051	999.63330730140	999.63330730140

uma derivada numérica com melhor precisão, notamos uma melhora até mesmo para os casos onde o h é o mesmo.

Já para valores de h muito **pequenos**, o erro também aumenta, mas agora como resultado da perda de precisão ao fazer operações com números cada vez menores e, em particular, por conta de erros de arredondamento (cancelamento subtrativo). Isso fica notório ao observar o salto de valores com respeito à segunda derivada de $h = 5^{-10}$ até $h = 5^{-12}$.

Com isso, podemos concluir que o valor ótimo de h está na região de $h = 5^{-6}$ até $h = 5^{-8}$, sendo o melhor quando $h = 5^{-8}$.

2 Tarefa 2 - Integração (ou Quadratura) numérica

2.1 Regras de aproximação de integrais

Para o cálculo numérico das integrais, utilizamos a subdivisão do intervalo de integração $[a, b]$ em N intervalos de tamanho h , com a integral sendo dada por

$$\begin{aligned} I &= \int_a^b f(x) dx \\ &= \int_a^{a+h} f(x) dx + \int_{a+h}^{a+2h} f(x) dx + \cdots + \int_{b-h}^b f(x) dx \end{aligned}$$

A fim de determinar as aproximações, recorreremos novamente à Série de Taylor (Equação 1):

Regra do Trapézio - $\mathcal{O}(h^2)$

$$I_T \approx h \left(\frac{1}{2}f(a) + f(a+h) + f(a+2h) + f(a+3h) + \cdots + f(b-h) + \frac{1}{2}f(b) \right)$$

Regra de Simpson - $\mathcal{O}(h^4)$

$$I_S = \frac{h}{3} (f(a) + 4f(a+h) + 2f(a+2h) + 4f(a+3h) + 2f(a+4h) + \cdots + f(b))$$

Regra de Boole - $\mathcal{O}(h^6)$

$$I_B = \int_{x_i-2h}^{x_i+2h} f(x) dx = \frac{2h}{45} (7f(x_{i-2}) + 32f(x_{i-1}) + 12f(x_i) + 32f(x_{i+1}) + 7f(x_{i+2}))$$

Observação: apenas para a Regra de Boole iremos iterar nas partições do intervalo. Para as demais, iremos utilizar a integral no intervalo $[a, b]$.

2.2 Resultados e conclusões

Nosso problema consiste em calcular numericamente a integral dada por

$$\begin{aligned} I &= \int_0^{2\pi} e^{-x} \cos(x) dx \\ &= \frac{1 - e^{-2\pi}}{2} \\ &\approx 0.49906627863 \end{aligned}$$

Já que o intervalo é $[0, 2\pi]$, então temos que o número de partições $N = \frac{(b-a)}{h} = 2\pi \cdot h^{-1}$. Com isso, na tabela do problema original temos na verdade o número de partições, de onde acharemos h .

Como podemos observar na Tabela 2, diferente do processo de derivação numérica, o cálculo da integral é um processo bem mais estável. Isso porque os cancelamentos subtrativos não ocorrem e, portanto, o erro é predominantemente dado pela quantidade de partições escolhidas. Há, porém, um limite de precisão da máquina ($\sim 10^{-15}$), que faz com que o valor não melhore de precisão mesmo que aumentemos o número de iterações (ou seja, o processo satura).

É possível notar isso avaliando os resultados para as regras do Trapézio e Simpson, onde a melhor precisão é dada para o maior valor testado. Para a regra de Boole, no entanto, ele atinge a precisão mais cedo e a partir daí satura e não melhora o resultado. Ou seja, observando os dados da tabela:

- Para a regra de Boole, a precisão da máquina é atingida logo cedo, por volta de $N = 2^9$
- Para a regra de Simpson, a precisão é atingida no último valor de $N = 2^{13}$
- Para a regra do Trapézio, notamos que ele ainda não atingiu o valor exato, o que indica que ainda precisaríamos de mais passos para melhorar o resultado

Tabela 2: Quadratura numérica para a integral $I = \int_0^{2\pi} e^{-x} \cos(x) dx$ por meio dos métodos do Trapézio, Simpson e Boole em função da partição $2\pi \cdot h^{-1}$.

$N = 2\pi \cdot h^{-1}$	h	Trapézio	Simpson	Boole
2^2	1.57079632679	0.71898458541	0.47932305694	0.45340198773
2^3	0.78539816340	0.55139754235	0.49553519467	0.49661600385
2^4	0.39269908170	0.51195877594	0.49881252046	0.49903100885
2^5	0.19634954085	0.50227715428	0.49904994706	0.49906577550
2^6	0.09817477042	0.49986822663	0.49906525075	0.49906627099
2^7	0.04908738521	0.49926671737	0.49906621428	0.49906627852
2^8	0.02454369261	0.49911638530	0.49906627461	0.49906627863
2^9	0.01227184630	0.49907880511	0.49906627838	0.49906627863
2^{10}	0.00613592315	0.49906941024	0.49906627862	0.49906627863
2^{11}	0.00306796158	0.49906706154	0.49906627863	0.49906627863
2^{12}	0.00153398079	0.49906647436	0.49906627863	0.49906627863
2^{13}	0.00076699039	0.49906632757	0.49906627863	0.49906627863
Exato	—	0.49906627863	0.49906627863	0.49906627863

```

1 | integral_0^2pi e^(-x)*cos(x) dx
2 | implicit none
3 |
4 | ! Intervalo de integração: [0, 2pi] -> tamanho = 2pi
5 | ! Então o número de intervalos é n = (b - a) / h = 2pi/h = 2pi * h^-1
6 |
7 | double precision integral_trapezio
8 | double precision integral_simpson
9 | double precision integral_boole
10 |
11 | double precision pi, h, a, b
12 | integer i, n
13 |
14 | open(unit=20, file="tarefa-2-integral-trapezio.dat")
15 | open(unit=30, file="tarefa-2-integral-simpson.dat")
16 | open(unit=40, file="tarefa-2-integral-boole.dat")
17 |
18 | pi = acos(-1.0d0)
19 |
20 | a = 0.0d0
21 | b = 2.0d0 * pi
22 |
23 | do i=2,13
24 |     n = 2**i
25 |     h = (b - a) / real(n, 8)
26 |     write(20, *) n, h, integral_trapezio(a, b, h, n)
27 |     write(30, *) n, h, integral_simpson(a, b, h, n)
28 |     write(40, *) n, h, integral_boole(a, b, h, n)
29 | end do
30 |
31 | close(20)
32 | close(30)
33 | close(40)
34 |
35 | stop
36 | end
37 |
38 | double precision function f(x)
39 | implicit none
40 | double precision x
41 | f = exp(-x)*cos(x)
42 |
43 | return
44 | end
45 |
46 | double precision function integral_trapezio(a, b, h, n)
47 | implicit none
48 | double precision a, b, h
49 | integer n
50 |
51 | double precision f
52 |
53 | integer i
54 | double precision x_i, soma
55 |
56 | ! Ajusta os passos do início e fim
57 | soma = 0.5d0 * ( f(a) + f(b) )
58 | do i=1,n-1
59 |     x_i = a + real(i, 8) * h
60 |     soma = soma + f(x_i)
61 | end do
62 |
63 | integral_trapezio = h * soma
64 |
65 | return
66 | end
67 |
68 | double precision function integral_simpson(a, b, h, n)
69 | implicit none
70 | double precision a, b, h
71 | integer n
72 |
73 | double precision f
74 |
75 | integer i
76 | double precision x_i, soma
77 |
78 | soma = f(a) + f(b)
79 | do i=1,n-1
80 |     x_i = a + real(i, 8) * h
81 |     if(mod(i, 2).ne.0) then
82 |         soma = soma + 4.0d0*f(x_i)
83 |     else
84 |         soma = soma + 2.0d0*f(x_i)
85 |     end if
86 | end do
87 |
88 | integral_simpson = h/3.0d0 * soma
89 |
90 | return
91 | end
92 |
93 | double precision function integral_boole(a, b, h, n)
94 | ! Como n é sempre múltiplo de 4
95 | ! (no nosso caso, pois fazemos com n = 4, 8, ...)
96 | ! podemos iterar por blocos de 4 subintervalos
97 | ! (ou seja, com 5 elementos)
98 | implicit none
99 | double precision a, b, h
100 | integer n
101 |
102 | double precision f
103 |
104 | integer i
105 | double precision soma, x0, x1, x2, x3, x4
106 |
107 | soma = 0.0d0
108 | do i=0, n-4, 4
109 |     soma = soma
110 |         + 7.0d0 * f( a + real(i, 8) * h )
111 |         + 32.0d0 * f( a + real(i+1, 8) * h )
112 |         + 12.0d0 * f( a + real(i+2, 8) * h )
113 |         + 32.0d0 * f( a + real(i+3, 8) * h )
114 |         + 7.0d0 * f( a + real(i+4, 8) * h )
115 |     end do
116 |
117 | integral_boole = 2.0d0*h/45.0d0 * soma
118 |
119 | return
120 | end

```

Figura 2: Código da tarefa 2.

3 Tarefa 3 - Raízes de funções

3.1 Métodos numéricos para o cálculo de raízes

O método para encontrar numericamente as soluções de uma equação, precisamos de duas etapas.

A primeira etapa consiste em realizar uma busca direta para eleger um intervalo onde há, garantidamente, ao menos uma raiz. Dado o intervalo $]x_-, x_+]$, isso pode ser feito observando se $f(x_-) \cdot f(x_+) < 0$, pois, se em algum lugar do intervalo a função mudou de sinal, então ela cruzou o eixo x e lá há ao menos uma raiz.

Ao encontrar o intervalo, podemos iniciar a segunda etapa e escolher algum dos seguintes métodos para iterar e procurar a raiz, apertando o intervalo e repetindo iterações até que o tamanho do intervalo seja menor que uma tolerância ϵ . Isto é, até que

$$|x_{+,n} - x_{-,n}| < \epsilon$$

Ao fim da execução do método, deslocamos o intervalo e continuamos a busca pelas raízes restantes.

Método da bissecção: Iteramos dividindo o intervalo ao meio e procurando de qual lado do intervalo ficou a raiz. Ou seja, com

$$x_m = \frac{x_{+,n} + x_{-,n}}{2}$$

verificamos se $f(x_m) \cdot f(x_{+,n}) < 0$. Se sim, então apertamos o intervalo para a direita, fazendo agora $x_{-,n+1} = x_m$ e $x_{+,n+1} = x_{+,n}$; caso contrário, apertamos o intervalo à esquerda, fazendo $x_{-,n+1} = x_{-,n}$ e $x_{+,n+1} = x_m$

Método de Newton-Raphson: Caso seja possível derivar a função facilmente (como por exemplo, no caso de polinômios), podemos utilizar as iterações como

$$x_{n+1} = x_n + \frac{f(x_n)}{f'(x_n)}$$

Método da secante: Quando a derivada não está disponível, podemos substituir a derivada no Método de Newton-Raphson pela derivada para trás de dois pontos:

$$x_{n+1} = x_n + f(x_n) \cdot \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}$$

3.2 Resultados e conclusões

Para o polinômio

$$f(x) = 27x^3 - 522x^2 + 3003x - 4508$$

temos que suas raízes são

$$\begin{aligned} x_1 &= \frac{7}{3} \approx 2.33333333 \\ x_2 &= \frac{23}{3} \approx 7.66666667 \\ x_3 &= \frac{28}{3} \approx 9.33333333 \end{aligned}$$

Tabela 3: Busca de raiz pelo método da Bissecção, Newton-Raphson e Secante para a raiz $x_1 = 7/3 \approx 2.3333333333$.

Iteração	Bissecção		Newton-Raphson	Método da Secante	
	$x_{i,-}$	$x_{i,+}$	x_i	x_{i-1}	x_i
$i = 0$	2.3000000000	2.3500000000	2.3000000000	2.4000000000	2.3340735614
$i = 1$	2.3250000000	2.3500000000	2.3329722699	2.3340735614	2.3333167526
$i = 2$	2.3250000000	2.3375000000	2.3333332903	2.3333167526	2.3333333374
$i = 3$	2.3312500000	2.3375000000	2.3333333333	2.3333333374	2.3333333333
$i = 4$	2.3312500000	2.3343750000	—	—	—
$i = 5$	2.3328125000	2.3343750000	—	—	—
$i = 6$	2.3328125000	2.3335937500	—	—	—
$i = 7$	2.3332031250	2.3335937500	—	—	—
$i = 8$	2.3332031250	2.3333984375	—	—	—
$i = 9$	2.3333007812	2.3333984375	—	—	—
$i = 10$	2.3333007812	2.3333496094	—	—	—
$i = 11$	2.3333251953	2.3333496094	—	—	—
$i = 12$	2.3333251953	2.3333374023	—	—	—
$i = 13$	2.3333312988	2.3333374023	—	—	—
$i = 14$	2.3333312988	2.3333343506	—	—	—
$i = 15$	2.3333328247	2.3333343506	—	—	—
$i = 16$	2.3333328247	2.3333335876	—	—	—
Exato	2.3333333333	2.3333333333	2.3333333333	2.3333333333	2.3333333333

Tabela 4: Busca de raiz pelo método da Bissecção, Newton-Raphson e Secante para a raiz $x_2 = 23/3 \approx 7.6666666667$.

Iteração	Bissecção		Newton-Raphson	Método da Secante	
	$x_{i,-}$	$x_{i,+}$	x_i	x_{i-1}	x_i
$i = 0$	7.6500000000	7.7000000000	7.6000000000	7.7000000000	7.6675630114
$i = 1$	7.6500000000	7.6750000000	7.6649897168	7.6675630114	7.6666540471
$i = 2$	7.6625000000	7.6750000000	7.6666655093	7.6666540471	7.6666666713
$i = 3$	7.6625000000	7.6687500000	7.6666666667	7.6666666713	7.6666666667
$i = 4$	7.6656250000	7.6687500000	7.6666666667	—	—
$i = 5$	7.6656250000	7.6671875000	—	—	—
$i = 6$	7.6664062500	7.6671875000	—	—	—
$i = 7$	7.6664062500	7.6667968750	—	—	—
$i = 8$	7.6666015625	7.6667968750	—	—	—
$i = 9$	7.6666015625	7.6666992187	—	—	—
$i = 10$	7.6666503906	7.6666992187	—	—	—
$i = 11$	7.6666503906	7.6666748047	—	—	—
$i = 12$	7.6666625977	7.6666748047	—	—	—
$i = 13$	7.6666625977	7.6666687012	—	—	—
$i = 14$	7.6666656494	7.6666687012	—	—	—
$i = 15$	7.6666656494	7.6666671753	—	—	—
$i = 16$	7.6666664124	7.6666671753	—	—	—
Exato	7.6666666667	7.6666666667	7.6666666667	7.6666666667	7.6666666667

```

1 ! f(x) = 27*x**3 - 522*x**2 + 3803*x - 4508
2
3 implicit none
4
5 double precision f
6
7 double precision eps ! tolerância para o método
8
9 ! Parâmetros para procura do intervalo com as raízes iniciais
10 double precision x_start, x_end, h ! 0 intervalo global para a busca de raízes e o passo dentro do intervalo
11 double precision x_a, x_b ! x_a = x_{-n} e x_b = x_{+n}, os limitantes do subintervalo na iteração
12
13 eps = 1.0d-6
14 x_start = -10.0d0
15 x_end = 10.0d0
16 h = 0.1d0
17
18 open(unit=20, file="tarefa-3-bisseccao.dat")
19 open(unit=30, file="tarefa-3-secante.dat")
20 open(unit=40, file="tarefa-3-newton-raphson.dat")
21
22 ! Encontrar intervalo onde o produto muda de sinal (onde há ao menos uma raiz)
23 do while(x_start.le.x_end)
24   x_a = x_start
25   x_b = x_start + h
26
27   ! Se no subintervalo há uma raiz, procuramos por ela
28   if ( ( f(x_a)*f(x_b) ).lt.0.0d0 ) then
29     call bisseccao(x_a, x_b, eps)
30     call secante(x_a, x_b, eps)
31     call newton_raphson(x_a, eps)
32   end if
33
34   ! Andamos com o intervalo para procurar mais uma
35   x_start = x_start + h
36 end do
37
38 close(20)
39 close(30)
40 close(40)
41
42 stop
43 end

```

```

45 double precision function f(x)
46 ! Função para encontrar as raízes
47 implicit none
48 double precision x
49 f = 27.0d0*x**3 - 522.0d0*x**2 + 3803.0d0*x - 4508.0d0
50 return
51 end
52
53 double precision function f_i(x)
54 ! Derivada primeira analítica de f, para o método de Newton-Raphson
55 implicit none
56 double precision x
57 f_i = 3.0d0*27.0d0*x**2 - 2.0d0*522.0d0*x + 3803.0d0
58 return
59 end
60

```

Figura 3: Código principal utilizado na tarefa 3.

Tabela 5: Busca de raiz pelo método da Bisseccção, Newton-Raphson e Secante para a raiz $x_3 = 28/3 \approx 9.3333333333$.

Iteração	Bisseccção		Newton-Raphson	Método da Secante	
	$x_{i,-}$	$x_{i,+}$	x_i	x_{i-1}	x_i
$i = 0$	9.3000000000	9.3500000000	9.3000000000	9.4000000000	9.3317166837
$i = 1$	9.3250000000	9.3500000000	9.3341947978	9.3317166837	9.3332564153
$i = 2$	9.3250000000	9.3375000000	9.3333338840	9.3332564153	9.3333334258
$i = 3$	9.3312500000	9.3375000000	9.3333333333	9.3333334258	9.3333333333
$i = 4$	9.3312500000	9.3343750000	—	—	—
$i = 5$	9.3328125000	9.3343750000	—	—	—
$i = 6$	9.3328125000	9.3335937500	—	—	—
$i = 7$	9.3332031250	9.3335937500	—	—	—
$i = 8$	9.3332031250	9.3333984375	—	—	—
$i = 9$	9.3333007812	9.3333984375	—	—	—
$i = 10$	9.3333007812	9.3333496094	—	—	—
$i = 11$	9.3333251953	9.3333496094	—	—	—
$i = 12$	9.3333251953	9.3333374023	—	—	—
$i = 13$	9.3333312988	9.3333374023	—	—	—
$i = 14$	9.3333312988	9.3333343506	—	—	—
$i = 15$	9.3333328247	9.3333343506	—	—	—
$i = 16$	9.3333328247	9.3333335876	—	—	—
Exato	9.3333333333	9.3333333333	9.3333333333	9.3333333333	9.3333333333

```

61      subroutine bisseccao(x_start, x_end, eps)
62      ! Método da bissecção:
63      !   Se o  $x_m \cdot x_{\{+,n\}} > 0$ , então o ponto médio caiu "à direita" da raiz,
64      !   indicando que devemos procurar "à esquerda" -> ( $x_{\{-,n\}}$ ,  $x_m$ )
65      !   Caso contrário, devemos procurar "à direita" -> ( $x_m$ ,  $x_{\{+,n\}}$ )
66      !
67      !   Obs.: Essa subrotina adiciona os dados no mesmo arquivo já aberto,
68      !   para facilitar o acompanhamento da busca do método.
69
70      implicit none
71
72      double precision x_start, x_end, eps
73      double precision f
74
75      double precision x_m      ! Ponto médio do intervalo da bissecção
76      double precision x_a, x_b ! Variáveis locais, para não modificar as variáveis fora do escopo
77
78      x_a = x_start
79      x_b = x_end
80
81      do while(1.eq.1)
82
83          x_m = 0.5d0 * (x_a + x_b)
84
85          if( f(x_m)*f(x_b).gt.0.0d0 ) then
86              x_b = x_m
87          else
88              x_a = x_m
89          end if
90
91          write(20, *) x_a, x_b
92
93          if(abs(x_b - x_a).lt.eps) then
94              exit
95          end if
96
97      end do
98
99      write(20, *) " "
100
101      return
102      end

```

Figura 4: Código da tarefa 3 que define o método da Bissecção.

```

104  subroutine secante(x_start, x_end, eps)
105  ! Método da secante:
106  !   Não utiliza derivada diretamente, mas sim três pontos para as iterações
107  ! (derivada para trás de dois pontos)
108  implicit none
109  double precision x_start, x_end, eps
110  double precision f
111
112  double precision x_next ! Terceiro ponto para o cálculo
113  double precision x_a, x_b ! Variáveis locais, para não modificar as variáveis fora do escopo
114
115  x_a = x_start
116  x_b = x_end
117
118  do while(1.eq.1)
119
120      x_next = x_b - f(x_b) * (x_b - x_a) / (f(x_b) - f(x_a))
121
122      x_a = x_b
123      x_b = x_next
124
125      write(30, *) x_a, x_b
126
127      if(abs(x_b - x_a).lt.eps) then
128          exit
129      end if
130  end do
131
132  write(30, *) " "
133
134  return
135  end

```

Figura 5: Código da tarefa 3 que define o método da Secante.

```

137      subroutine newton_raphson(x_start, eps)
138      ! Método de Newton-Raphson
139      !     Utiliza a derivada analítica para as iterações
140      implicit none
141      double precision x_start, eps
142      double precision f, f_i
143
144      double precision x_a, x_b
145
146      x_a = x_start
147
148      do while(1.eq.1)
149          x_b = x_a - f(x_a) / f_i(x_a)
150          write(40, *) x_a, x_b
151          if(abs(x_b - x_a).lt.eps) then
152              exit
153          end if
154          x_a = x_b
155      end do
156
157      write(40, *) " "
158      return
159      end
160

```

Figura 6: Código da tarefa 3 que define o método de Newton Raphson.